Error Detection and Correction

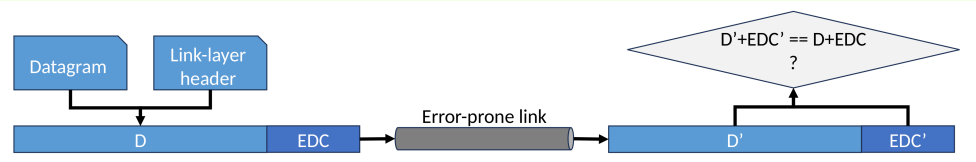
Lecture 26

Computer Networks and Distributed Programming

Nothing says the bits are the ones that were sent

- The receiver can now find where a frame begins and where it ends. That is all it can do so far.
- With the payload alone there is nothing to compare it against: every sequence of bits is a legitimate sequence of bits.
- So the sender computes something redundant over the data and sends it along, in the trailer at the end of the frame.
- The receiver repeats the computation on what arrived, and compares the two.
- Today: what that redundant something is, how strong it is, and what happens when it is not strong enough.

D and EDC



- D: the data to protect — not only the datagram, but the link-layer header too: addresses, sequence numbers, every field.
- EDC: the error-detection and -correction bits, computed over D before it leaves.
- What arrives is D' and EDC', which may differ from either. The receiver recomputes the EDC over D' and compares it with EDC'.

Detected, not occurred

The check succeeds. Does that mean no bit was flipped?

No. It means no error was *detected*. Some patterns of flips leave the redundant bits perfectly consistent with the damaged data.

- The question a receiver answers is always the first one, and the wording matters: errors are detected, not observed.
- A stronger scheme makes the probability smaller. No scheme makes it zero, and the pathological cases are what the rest of the lecture is about.
- And strength is paid for: more bits to compute, more bits to transmit, more circuitry to compute them with.

Two strategies for the same problem

Both add redundancy. They differ in how much

- Error-detecting codes: enough redundancy to know that something is wrong, and ask for it again.
- Error-correcting codes: enough to work out what must have been sent, and repair it on the spot.

- The second is called forward error correction: forward, because nothing travels back.
- The choice is not about which is stronger. It is about what a retransmission costs on that particular link.

Each strategy has its channel

- Fibre, coax, twisted pair: errors are rare. Detecting them and resending the occasional bad frame is cheaper than carrying repair bits on every frame.
- Wireless, ageing local loops: errors are the norm. And a retransmission is as likely to be damaged as the first attempt was.
- So on a noisy channel one repairs locally, and on a clean one one asks again. Both are in use, on links a metre apart.
- Neither handles everything: the channel does not treat the check bits differently from the data bits. They are all just bits to it.
- The code must therefore be strong enough for the errors that are expected on that link.

A single parity bit

- One bit added to d data bits, chosen so that the number of ones among the $d + 1$ is even — or odd, if that is the convention agreed.
- Equivalently: the parity bit is the XOR of all the data bits. One bit of overhead, however long the message.

Message	Parity bit (even)	Parity bit (odd)
1 0 1 0 1 1 0 0	0	1
1 1 0 1 0 0 0 0	1	0

The check at the receiver

Even scheme, one message sent twice

- Received `1 0 1 0 1 1 0 0` with parity 0: four ones, the count is even. Accepted.
- Received `1 0 1 1 1 1 0 0` with parity 0: the fourth bit was flipped in transit. Five ones, the count is odd. Rejected.

- The receiver needs no memory of what was sent, and no second copy to compare against: it counts, and the count either fits the convention or it does not.
- Which is why one bit of hardware does it, at the speed the bits arrive.

Only an odd number of errors

What if two bits of the same frame are flipped?

The number of ones changes by two, so its parity does not change at all. The count fits, the frame is accepted, and the damage is passed upwards.

- A single parity bit detects an odd number of flips, and nothing more. Two, four, six: invisible.
- In the language we will use at the end of the lecture, this is a code of distance two: enough to notice one flip, not enough to promise anything about two.
- Whether that is acceptable depends entirely on how errors actually arrive on a link.

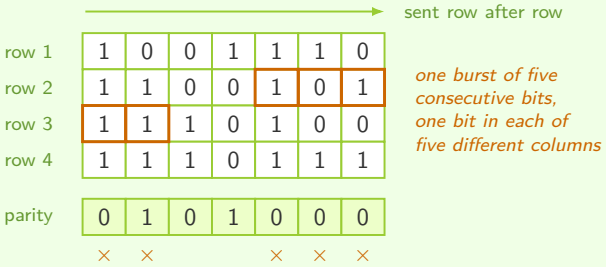
Errors arrive in bursts

- If flips were independent and rare, several of them in one frame would be so improbable that one parity bit would do.
- They are not: an error burst is a run whose first and last bits are wrong, with a mixture of wrong and correct bits between them.
- The physical cause lasts longer than one bit time: a fade on a radio channel, a transient on a copper line, a scratch on a disc.
- Under burst conditions the probability that single-bit parity misses the damage approaches one half.
- The other side of it: with 1000-bit blocks and one error in a thousand bits, independent errors damage most blocks — bursts of a hundred damage one block in a hundred.

And still, one parity bit is cheap

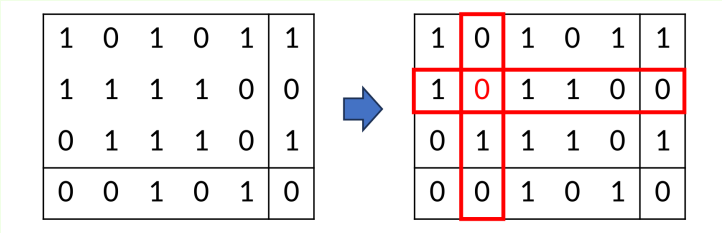
- Take a wired cable good enough to flip one bit in a million, and blocks of a thousand bits.
- To repair a single error: about ten check bits per block are needed — ten thousand bits per megabit of data.
- To notice it: one bit per block, a thousand per megabit; one block in a thousand is found bad and sent again, at 1001 bits.
- Two thousand bits against ten thousand, and that is the whole argument: on a good link, detection with retransmission wins.
- Make the link bad and the arithmetic reverses, because the retransmissions multiply and each one can fail too.

Spreading a code over the transmission order



- The idea: compute the check bits over the data in a different order from the one in which the bits go out.
- A burst that spans n transmitted bits touches at most one bit per column, and every column it touched fails its parity.
- The parity has not become stronger. The arrangement has: a code for isolated errors now catches bursts.

Two-dimensional parity

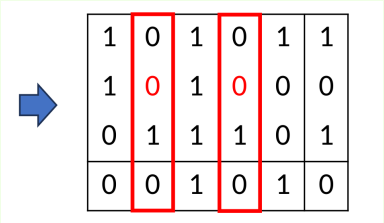


- The d bits are laid out in i rows and j columns, with a parity bit for every row and every column: $i + j + 1$ in all.
- One flipped bit breaks exactly one row and exactly one column, and the crossing of the two names the bit.
- Naming it is enough to flip it back: detection has become correction, with no message sent back.

Two errors in the same row

Two bits of one row are flipped. What does the receiver see?

The row parity is untouched — two flips cancel — and only the two columns complain. The frame is known to be damaged, and no bit can be named.



1	0	1	0	1	1
1	0	1	0	0	0
0	1	1	1	0	1
0	0	1	0	1	0

- So the scheme detects any two errors and corrects only one. The distance between the two verbs is the subject of the end of this lecture.

Forward error correction

- Repairing at the receiver has a name: forward error correction, FEC. The frame is used as it arrived, and nothing is asked again.
- The gain is not the bits saved: it is the round trip not waited for — and on some links that wait is the whole problem.
- Real-time audio and video, where data that is late is worse than data that is wrong. Deep-space links, where a round trip is measured in minutes.
- Optical discs, where the damage is a scratch and there is nobody to ask for a second copy.
- It also combines with retransmission: repair what can be repaired, ask again for the rest.

Checksums

- The second family: read the data as a sequence of k-bit integers and add them up.
- You have met one already, at the transport layer: sixteen-bit words, the ones' complement of the sum, carried in the segment header.
- Cheap and weak: a sum does not notice reordered words, zeros added or removed, or two packets spliced together — and buggy hardware produces exactly those.
- Fletcher's variant: adds each word multiplied by its position to the running sum, so moving a word changes the result.
- It is still a sum, and the link layer wants something better.

A sum above, a division here

- The question: why does the transport layer add, while the link layer divides, over data that is largely the same?
- Transport runs in software, inside the operating system, on the host's own CPU: whatever it computes runs on every segment of every connection.
- The link check runs in the adapter, in dedicated circuitry, on one frame at a time and at wire speed.
- A division costs almost nothing there, and a great deal in a kernel loop.
- So the strength of a check follows where it is computed, not how precious the data is.

Cyclic redundancy check

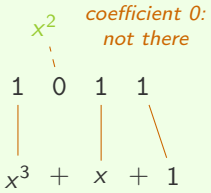
CRC

Sender and receiver agree in advance on a generator G of $r + 1$ bits, whose leftmost bit is 1. The sender appends r bits to the data D so that the whole $d + r$ pattern is divisible by G, modulo 2.

- The receiver divides the $d + r$ bits it received by the same G. Remainder zero, the frame is taken; anything else, an error.
- Modulo-2 arithmetic: no carries, no borrows. Addition and subtraction are the same operation, and both are XOR.
- So the division is a sequence of alignments and XORs: G is placed under the leftmost 1 left in the string, and XORed there.

Bit strings are polynomials

- A string of k bits is read as the coefficients of a polynomial of degree $k-1$: the leftmost bit is the highest degree.
- The coefficients are 0 and 1, and the arithmetic is done modulo 2 — which is why sum and difference coincide.
- So D becomes $D(x)$, the generator becomes $G(x)$ of degree r, and appending r zeros to D is multiplying $D(x)$ by x^r.



x	y	$x \oplus y$
0	0	0
0	1	1
1	0	1
1	1	0

Why appending the remainder works

The identity behind the method

Divide $x^r D(x)$ by $G(x)$, and call the quotient $Q(x)$ and the remainder $R(x)$:

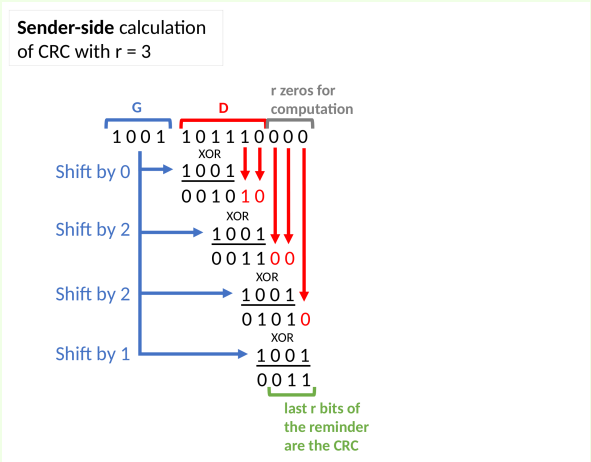
$$x^r D(x) = Q(x)\, G(x) + R(x), \qquad \deg R < r.$$

Then $M(x) := x^r D(x) - R(x) = Q(x)\, G(x)$ is divisible by $G(x)$ by construction, and M is what goes on the wire.

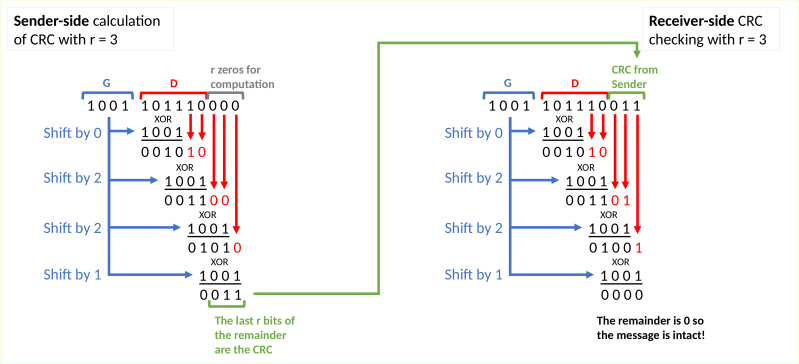
- Subtracting is XORing, and R has at most r bits: so M is simply D with R written into the r zeros that were appended.
- The CRC is the remainder: nothing is invented — the check bits are whatever the division leaves over.

Computing the CRC

- $d = 6$, $D = 101110$, $r = 3$, $G = 1001$: agreed before any frame is sent.
- Three zeros are appended — that is the multiplication by x^r — and the division begins.
- At every step G is aligned with the leftmost 1 still standing, and XORed into the string.
- The last r bits left over are the CRC: 011. Nine bits go out, 101110 011.



Checking at the receiver



- The receiver divides everything it received, the CRC bits included, by the same G.
- Remainder zero: the frame is taken. Anything else: an error, and the division does not say where it is.
- Zero means accepted, which is not the same as intact.

What a CRC catches

- Write what arrives as $T(x) + E(x)$: every 1 in the error polynomial $E(x)$ is a bit that was flipped in transit.
- $T(x)$ divides by $G(x)$, so the remainder the receiver computes depends on $E(x)$ alone: an error escapes only if $G(x)$ divides it.
- One flipped bit: $E(x) = x^i$, and a generator with two or more terms never divides that. All single errors are caught.
- An odd number of flips: if $x + 1$ is a factor of $G(x)$, every one of those is caught — statistically half of all cases.
- Any burst of r bits or fewer: it is shorter than the generator, so the remainder cannot come out zero.

The standards

- Generators are standardised at 8, 12, 16 and 32 bits, and a protocol names the one it uses. Nobody invents a generator.
- CRC-32: adopted by several IEEE link-layer protocols, 33 bits long:

$$100000100110000010001110110110111$$

- Every burst shorter than $r + 1$ bits is detected with certainty; a longer one escapes with probability 0.5^r, so it is caught with probability $1 - 0.5^r$.
- With $r = 32$ that is one damaged frame in four thousand million slipping through, on top of everything caught for certain.
- These four bytes sit at the end of every Ethernet frame. We open one, field by field, at the local network.

When the check fails

The CRC does not match. What does the adapter do with the frame?

It drops it, and says nothing — not to the sender, not to the network layer above. On Ethernet nobody is told at all.

- So a damaged frame becomes a lost frame, indistinguishable from one that was never delivered.
- Recovery falls to whoever above is keeping a timer: the transport layer, or the application, seconds later and an entire round trip away.
- Which is exactly the price that correcting on the spot avoids — and on a link where this happens often, the price is worth avoiding.

Codewords

- A code turns m data bits into $n = m + r$ bits. The n bits are a codeword, and one speaks of an (n, m) code.
- Rate: the fraction m/n that carries information: about a half on a noisy channel, close to one where the channel is good.
- Because the check bits are computed from the data, only one string in 2^r is a legal codeword. The rest cannot be produced by a sender.
- That sparseness is the whole mechanism: a received string that is not a codeword can only be damage.
- 4B/5B is exactly this: four data bits sent as five, sixteen symbols out of thirty-two never produced — which is where the frame delimiters came from.

Hamming distance

Hamming distance

The number of bit positions in which two words differ — equivalently, the number of flips needed to turn one into the other. It is the number of ones in their XOR.

Example

$W_1 = 00110110, \quad W_2 = 00011010, \quad d(W_1, W_2) = 3.$

- The distance of a code is the smallest distance between any two of its codewords: the closest pair is the one that decides what the code can promise.

What the distance buys

A code of minimum distance L

detects up to $L-1$ errors, and corrects up to $\lfloor (L-1)/2 \rfloor$.

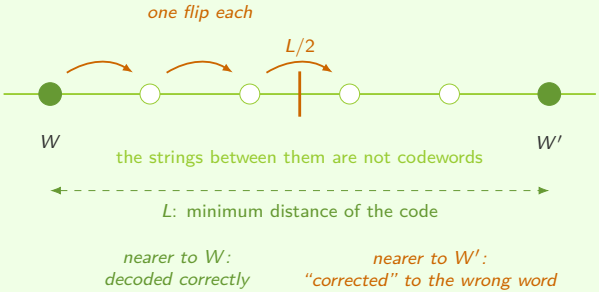
- To detect d errors, distance $d+1$: with that much room, d flips can never land one codeword on top of another: an illegal string arrives, and illegal means damaged.
- To correct d errors, distance $2d+1$: the codewords are far enough apart that after d flips the word sent is still the nearest one.
- A code cannot do both at its limit: the same distance is spent once, and one decides what to spend it on.

Four codewords of ten bits

	Codeword	Data
W_1	0000000000	00
W_2	0000011111	01
W_3	1111100000	10
W_4	1111111111	11

- Two bits of information in ten: a rate of $1/5$, deliberately extravagant, so that the arithmetic is visible.
- Any two of the four differ in at least five positions, so $L = 5$: four errors detected, two corrected.
- W_2 is sent and 0100011101 arrives: two bits flipped. It is not one of the four, so the damage is visible at once.

Decoding is the nearest codeword



- A string that is not a codeword is assigned to the codeword closest to it, on the assumption that few flips are likelier than many.
- That is right while the flips stay under half the minimum distance; past the halfway mark the repair makes things worse.
- 0100011101 is two flips from W_2 and five or more from every other: it is decoded as W_2.

Hamming codes

- Searching all the codewords for the nearest one is fine with four of them. With a real code it is a search nobody can afford per frame.
- Hamming's construction: number the bits from 1 and put the check bits in the positions that are powers of two; each data bit is covered by the check bits appearing in its position number.
- The receiver redoes every check. The ones that fail, read together as a binary number, are the position of the flipped bit — the syndrome.
- No search: the arithmetic hands over the answer. The $(11, 7)$ code protects seven data bits with four check bits.
- Its distance is three, so it corrects one error or detects two — the choice of the previous frame, made concrete.

What real links use

- Convolutional codes: no block boundary at all: each output bit depends on the last few input bits, and decoding looks for the most likely input. In 802.11, and on the Voyager probes since 1977.
- Reed-Solomon: works on symbols rather than bits, so a burst inside one symbol counts as a single error. CDs, DVDs, DSL, satellite links.
- LDPC: each check bit covers only a few data bits; excellent on large blocks. Ten-gigabit Ethernet and the recent 802.11.
- The advice that comes with them: error codes are applied mathematics. Take one from a standard; do not invent your own.

Summary

- The sender computes redundant bits over the frame and the receiver recomputes them: the answer is whether an error is detected, never whether one occurred.
- Parity: one bit, and only an odd number of flips — while real errors come in bursts. Laid out in two dimensions it names a single flip and repairs it.
- Checksums: sums, cheap and weak, computed in software above. CRC: a division, strong, computed in the adapter's hardware.
- A CRC catches every single error, every odd number of them, and every burst up to r bits; the frame that fails is dropped, and the layers above find out late.
- Correction lives on distance: $L - 1$ errors detected, $\lfloor (L-1)/2 \rfloor$ corrected — worth its bits where a round trip costs more than they do.
- So far a link has been two machines and a wire. Next: many machines, one wire, and the rules for who speaks.